

Candidate Assignment: Build a Net-New Long-Horizon Hard-but-fair Task

Goal: Create one fair task that exposes a substantive failure in GPT-5.5-high or Claude Opus 4.7 with their native harnesses.

We care about how you approach the problem, make reasonable assumptions and resolve the ambiguity. You will be evaluated on those dimensions more than if you got to the right solution. We are looking for high-agency, high-ownership people who have a bias for action. P.S. We have seen 1000+ assignments so please don't one shot this, we know how Claude thinks, we want to see how you think :)

1. Objective

Create one original, long-horizon agent task in Harbor format. The task should be difficult because it requires planning, inspection, state tracking, debugging, verification, and iterative tool use, not because of brittle formatting, missing dependencies, vague instructions, or hidden tricks. The target outcome is a clean, reproducible, evidence-backed failure from GPT-6-astra-high or Claude Fable 5.1, while the task remains solvable by the included oracle solution. It should also be a task which has some connection to reality.

2. What “fairly fails” means

A fair failure is:

Solvable: A competent human or strong long-horizon agent can solve it from the provided files and instructions. The included oracle solution must pass.

Unambiguous: The instruction says what final artifact or state is required, what constraints matter, and what success looks like.

Substantive: The model fails because of a real capability gap: multi-step reasoning, tool-use discipline, data/code forensics, planning under constraints, environment understanding, or verification discipline.

Reproducible: A fresh checkout builds and runs without private machine state. Any network, secrets, model calls, or special resources are explicitly declared.

Non-brittle: The verifier checks meaningful task outcomes, not superficial strings or formatting artifacts. Disqualifying unfair failures include impossible tasks, missing dependencies, broken Dockerfiles, random network behavior, hidden requirements, copied benchmark tasks, leaked solutions, brittle exact-string grading, and failures caused only by too-short time limits.

3. Long-horizon bar

A good task should require several linked steps. It should not be solvable by one obvious command or by reading a single small file. It should include at least three meaningful subgoals, realistic seed artifacts, hypothesis formation, verification, and a durable final deliverable such as a code patch, repaired dataset, analysis report, configured service, migrated state, or validated workflow.

4. Harbor submission requirements

Submit a zip file or repository containing exactly one task directory unless otherwise approved.

Use a stable slug such as `collinear-candidate/<task-name>`. Required contents:

- `instruction.md`: clear task prompt, constraints, files to inspect, and expected deliverable.
- `task.toml`: Harbor metadata, timeouts, resource requirements, network policy, authorship, and category.
- `environment/Dockerfile` or `environment/docker-compose.yaml`: pinned, reproducible environment.
- `tests/test.sh`: verifier entrypoint that writes `/logs/verifier/reward.txt` or `/logs/verifier/reward.json`.
- `tests/test_*.py` or grader files: functional checks, fixtures, metrics, or judge logic.
- `solution/solve.sh`: oracle solution that passes the verifier.
- Seed files: codebase, logs, datasets, configs, mock services, docs, schemas, or databases as needed.
- `README.md` or `RUN_REPORT.md`: task idea, fairness rationale, why this is an economically viable/real task, model runs, verifier design, limitations, and reproduction commands.

Minimum commands to include:

```
harbor run -p ./<task-dir> -a oracle
```

```
harbor run -p ./<task-dir> -a <agent> -m <gpt-5.5-high-or-opus-4.7>
```

```
harbor view ./jobs
```

5. Verifier requirements

The verifier must check the core success criteria. You can use programmatic verifiers or agentic judges. Please avoid LLM as a Judge judging a trajectory. For multi-criterion grading, prefer

`reward.json`, for example:

```
{  
  "overall": 0.0,
```

```
"functional_correctness": 0.0,  
"constraint_satisfaction": 0.0,  
"robustness": 0.0,  
"artifact_quality": 0.0  
}
```

Make sure that your failure is not a false positive or negative.

6. Net-new requirement

The task must be newly created for this exercise. No direct ports of SWE-bench, Terminal-Bench, public CTFs, Kaggle notebooks, coding challenges, public benchmark tasks, public issues, or previous Collinear tasks. Open-source libraries and small public datasets are allowed when properly licensed and transformed into a genuinely new task scenario. Include a short provenance note explaining what was created from scratch and what external sources were used.

Feel free to take any domain of your choice: Coding, Finance, MCP Tool Calling, <Literally anything>

7. Evidence of target-model failure

Run GPT-5.5-high or Claude Opus 4.7 (you can use harbor on prem) using the codex or claude code harness. Feel free to contact to adit@collinear.ai for API credits. A strong submission includes 1 model trials or clear pass/fail logs with the model failing on atleast one critical thing. Include a short failure analysis: wrong hypothesis, incomplete inspection, bad state tracking, brittle patch, skipped verification, report hallucination, or other substantive failure.

8. Scoring rubric

- Harbor compliance and reproducibility: 20 points
- Fairness and solvability: 20 points
- Verifier quality: 20 points
- Long-horizon difficulty: 15 points
- Evidence of frontier-model failure: 15 points
- Originality and realism: 10 points

9. Final checklist

- I submitted exactly one net-new Harbor task directory.
- `instruction.md` is clear and does not leak the solution.

- `task.toml` has accurate metadata, timeouts, resources, and network policy.
- The environment builds from a clean machine or documented container provider.
- `solution/solve.sh` is executable and oracle passes.
- `tests/test.sh` writes `/logs/verifier/reward.txt` or `reward.json`.
- The verifier checks meaningful success criteria and rejects shallow solutions.
- `RUN_REPORT.md` includes oracle result, target-model result or run command, failure analysis, realism/economic viability of task, and fairness audit.
- External sources and licenses are documented.
- The task is not copied from an existing benchmark, issue, CTF, tutorial, or prior internal task.

Reference links

- Collinear public context: <https://x.com/CollinearAI/status/2057593797455032802>
- Harbor documentation: <https://www.harborframework.com/docs>
- <https://x.com/neversupervised/status/2075432858270003462>

P.S. - Brainstorm more and think about it - we do care about creativity a lot!!